# University of West Attica
# Department of Informatics and Computer Engineering

## Digital Systems Design

### Academic year 2022-2023
### Semester Course Exercise

**Delivery date:** 16 /6/202 3

The purpose of this work is to design and implement a single-cycle MIPS processor. The processor has the reset and clock signals as inputs , and no output. The reset signal drives the PC unit to the value 0. It also drives the Register file and zeroes all registers .

The processor includes the following modules (numbering is shown in Figure 1)

| | |
|---|---|
| 1 | ALU |
| 2 | Register file |
| 3 | Data memory |
| 4 | Instruction memory (will be implemented as ROM ) |
| 5 | Control unit |
| 6 | ALU control unit |
| 7 | PC |
| 8 | 5-way 2-in-1 multiplexer |
| 9 | 16-to-32 sign expansion unit |
| 10, 11, 12 | 32-way 2-in-1 multiplexer |
| 13 | Shift unit left by 2 (32- bit ) |
| 14,15 | bit adder |
| | 2-input AND gate |

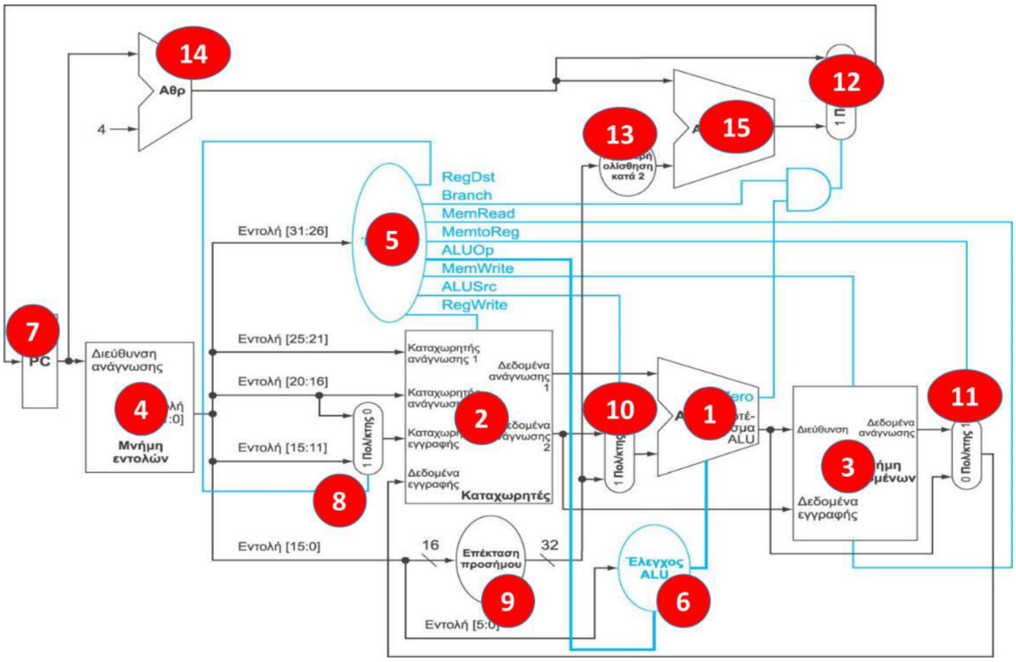


**Figure 1** : Processor Implementation

The processor will execute the commands: add , sub , addi , lw , sw , **bne** (note, the processor structure in Figure 1 may need to be slightly modified)

For ease of implementation, consider that:
- The instruction memory has 16 locations (of 32 bits )
- The data memory has 16 locations (of 32 bits )
- The register file has 16 locations (of 32 bits )
- After each instruction is executed, the contents of PC are incremented by 1.

Implement the following:

**1. Entity MIPS**
Create an entity MIPS .

**2.** Load the following instructions into instruction memory, starting at location 0.
```
addi $0, $0, 0 # not needed ( registers are zeroed)
addi $2, $2, 0 # not needed ( registers are zeroed)
addi $2, $4, 0 # not needed ( registers are zeroed)
add $3, $0, 1
add $5, $0, 3
L1: add $6, $3, $0
    sw $6.0($4)
    add $3, $3, 1
    add $4, $4, 1
    add $5, $5, -1
bne $ 5,$ 0, L 1
```

**Attention:** The placement of the commands must be done before the units are Compile and Loaded .

**3. MIPS architecture development**
Create an architecture that will include all the previous elements as components , appropriately connected.

**Note: for convenience, you can implement the instruction and data memory as a 32-** bit array . In this case:
- The adder (14) will add the number 1 to the contents of the PC and not the number 4.
- The left shift unit by 2 (13) can be omitted.

**4. Function control**
**4.1** Activate the reset signal for (a little less than) one cycle.
**4.2** Run the program. The simulation should show the values of the registers and memory locations.

**Submission Instructions**

**Delivery method:** Submit zip file to the "Projects" folder, Semester 2022-2023

The following files should be included in the work deliverables.

| AA | Description | File name |
|---|---|---|
| 1 | Document doc ( x ) that will contain<br>• introductory page (course & student details).<br>• circuit description<br>• circuit code and testbench ,<br>• screenshot from the execution showing the values of the registers and the contents of the memory locations that change value. | AM _ SURNAME _01_DESCRIPTION.DOC(X) |
| 2 | The circuit code in a separate file | AM_ADJECTIVE_02_ MIPS . vhd |
| 3 | testbench code in a separate file | AM_ADJECTIVE_0 3 _ testbench . vhd |

The three files will be compressed into a file named AM _ EPITHETO . zip

The surname will be written in Latin characters.

**Attention:** will not be graded and will be reset to zero:
- Code that does not pass the compile phase .
- Work that will not be accompanied by the necessary documentation
- Work that will not include a testbench